Are NBFNet's Path Explanations Actually Faithful?

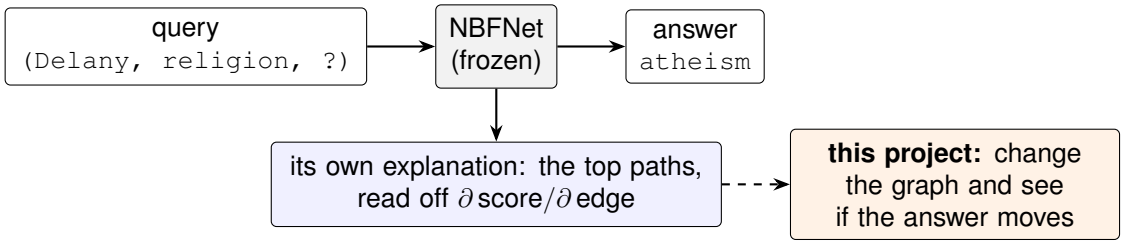
A deletion and edit benchmark on a frozen knowledge graph reasoner

Shadi Khodagholi

CAP 6938 AI Knowledge Systems · Research Track · UCF

`github.com/shadi97kh/nbfnet-path-faithfulness`

What this project does



- The paths are sold as the **provenance** of the answer. That is the whole case for paths over embeddings.
- A gradient says what happens if an edge changes *a little*. Whether the model *relied* on it is a question about removing it.
- So I audit the released explanation code by **intervening**, not inspecting: Topic 8's lesson, on a substrate where the edit is exact.

Five interventions, on the graph itself

	what changes in the graph	the question it answers
A	nothing; call NBFNet's `visualize()`	what does the method claim?
B	delete each shortlisted path, one at a time	does its ranking match real effect?
C	delete the whole shortlist, or keep only it	necessary? sufficient?
D	point the last hop at a different entity	does the answer follow the edit?
E	turn one relation channel's gain from 1 to 0	does the gradient predict the dose?

- Every intervention removes or rewires real edges and reruns the model. No masks, no approximations, **no training**: the released checkpoint, every parameter frozen.
- Controls: the same number of **random** edges; the **bottom** of the method's own candidate list; the tail entity's **degree**; whether the query's **reverse fact** is already in the graph.

The benchmark

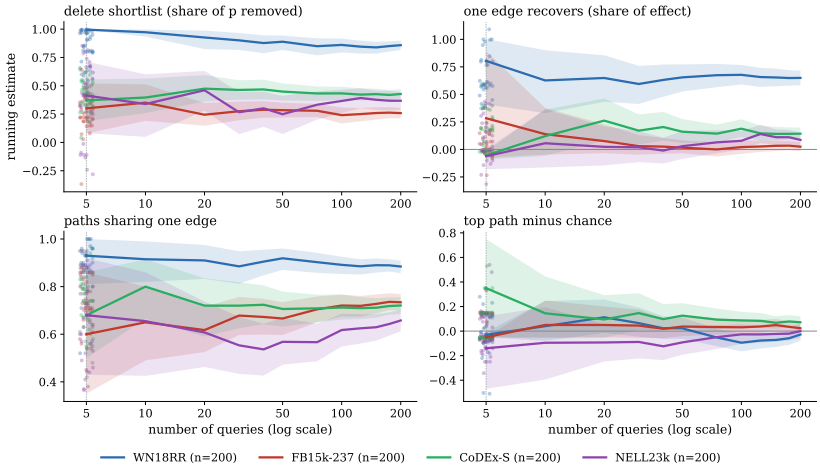
graph	queries	MRR	
WN18RR	200	0.484	in pre-training
FB15k-237	200	0.371	in pre-training
CoDEx-S	200	0.464	held out
NELL23k	200	0.258	held out

- 800 queries, each with 20 candidate paths
- 15,447 single-path deletions, plus joint deletions, keep-only, random controls and rewires
- MRR on the **full** test set of each graph, with ULTRA's own evaluation code

Design choices that matter

- Queries drawn in a **fixed random order**, set before anything was measured. No picking.
- Only queries the model ranks in its top ten: explaining a mistake is a different question.
- Two graphs the model was trained on, two it never saw.
- Every interval is a 95% bootstrap over queries. The model is deterministic, so queries are the only source of variance.

Is that enough queries? The data answers it



Dots at the left: every disjoint block of five queries. On WN18RR, a five-query study of "does the top path beat chance" could have reported anything from -0.51 to 0.33: either sign. At full size, 194 queries with at least two paths, the interval is ± 0.052.

Live demo

```
python -m pathfaith.demo --n 5
```

- Loads the frozen checkpoint on CPU and picks five queries it gets right, on CoDEx-S, a graph it never saw, with real Wikidata names.
- For each: prints NBFNet's own explanation, then runs every intervention **live**, one forward pass per line.
- Watch the **random-edge control** stay at zero, the **busiest edge** line, and the **neuromodulate** line: one relation channel turned down to half and to zero, against what the gradient predicted.
- It ends by reading the full benchmark off disk, so the five live queries are visibly one slice of 800.

Then again with `--seed 7`: a different draw of queries, to show it is not one lucky run. A recording of both runs is the fallback.

Result 1: the edges it picks are load-bearing

	WN18RR	FB15k-237	CoDEx-S	NELL23k
delete shortlist, share of *p* removed	0.858	0.259	0.427	0.368
95% CI	[0.813, 0.898]	[0.212, 0.303]	[0.386, 0.470]	[0.290, 0.444]
delete as many random edges	0.000	0.000	0.000	0.007
answer falls out of top 10	76.0%	17.0%	32.5%	31.0%
keep only shortlist: answer *gains p*	8.0%	36.5%	81.0%	12.0%

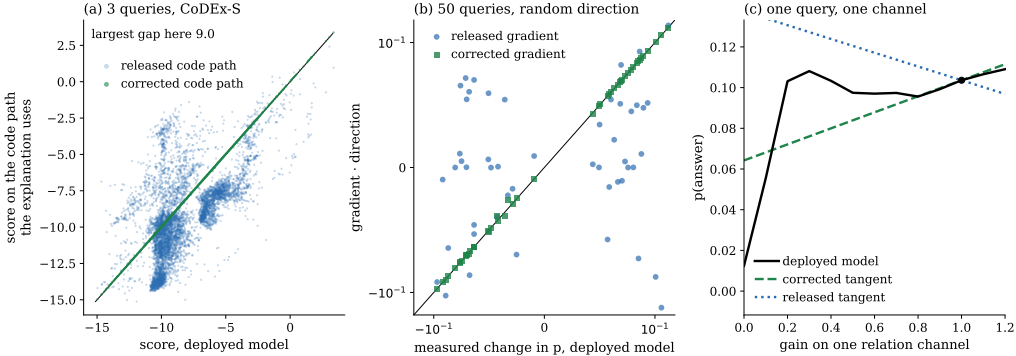
- On every graph, deleting the explanation hurts and deleting random edges does not. The gradient is pointing at something real.
- Keep-only is not a clean test on a graph: strip it to a few edges and the rival answers vanish too, so *p* can go *up*. I report it and draw nothing from it.

Result 2: the order it puts them in is not

	WN18RR	FB15k-237	CoDEx-S	NELL23k
top path is the most damaging	0.253	0.080	0.125	0.306
chance rate (ties count)	0.280	0.057	0.053	0.307
Kendall τ, rank vs. effect	-0.009	0.048	0.076	0.041
95% CI	[-0.054, 0.037]	[0.010, 0.085]	[0.043, 0.108]	[-0.023, 0.102]

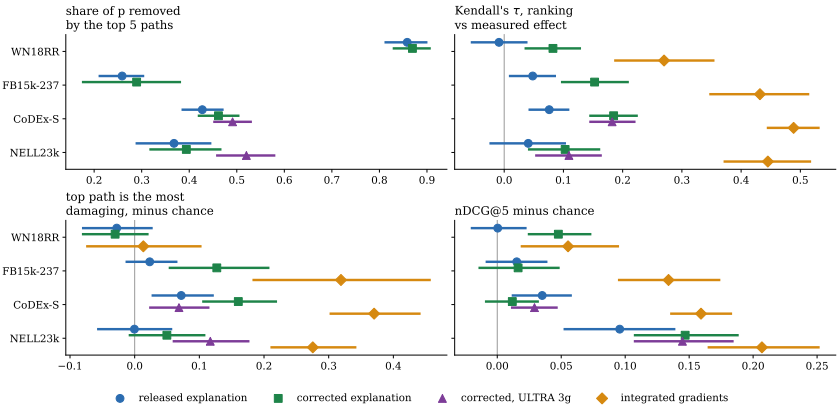
- A faithful ranking would give τ near 1. As released, it is near zero everywhere, and on WN18RR the top path is the most damaging one *less* often than a coin would pick it.
- This is Hase et al.'s Topic 8 result one domain over: what the method locates is not what predicts the effect of changing it.

Trying to fix the order, I found a bug in the explanation code



The fast kernel the model runs on sends messages one way along each edge; the generic code `visualize()` falls back to, to get a gradient, sends them the other way. So the explanation differentiates a **different model**: scores up to 10.5 logits off, gradient off by as much as the effect (median error 100%), wrong sign on 21 of 50 directions. Fix: run it on the reversed graph. Error then 0.7%. NBFNet-PyG has the same code pattern (read, not run).

Fixing it helps a little. Integrated gradients help more. Neither is enough.



One variable at a time. Direction fix: Kendall τ up by 0.109 on CoDEx-S and 0.081 on NELL23k. Integrated gradients on the same candidates: τ from 0.270 to 0.489 on the four graphs. The ULTRA paper's own checkpoint (MRR 0.472 vs. 0.472 reported) gives the same picture.

Result 3: one edge or many depends on the graph

	WN18RR	FB15k-237	CoDEx-S	NELL23k
paths sharing the busiest edge	0.884	0.734	0.721	0.658
distinct edges / edges listed	0.248	0.484	0.512	0.295
that one edge recovers	0.650	0.025	0.143	0.086
reverse fact already in graph	57.5%	0.0%	13.0%	4.5%

- On WN18RR the list of paths is mostly one edge written out many ways, and much of that is queries a one-hop reverse lookup answers, served as up to twenty paths of up to six hops.
- On denser graphs the paths are genuinely different. The explanation degenerates where the task is degenerate, and **nothing in its output tells you which case you are in**.

Relation channels as neuromodulators

Bull et al., NeurIPS 2025: a few broadcast modulators multiply a circuit's connections,

$$W_{\text{eff}} = W_0 + \sum_k T_k \, n_k,$$

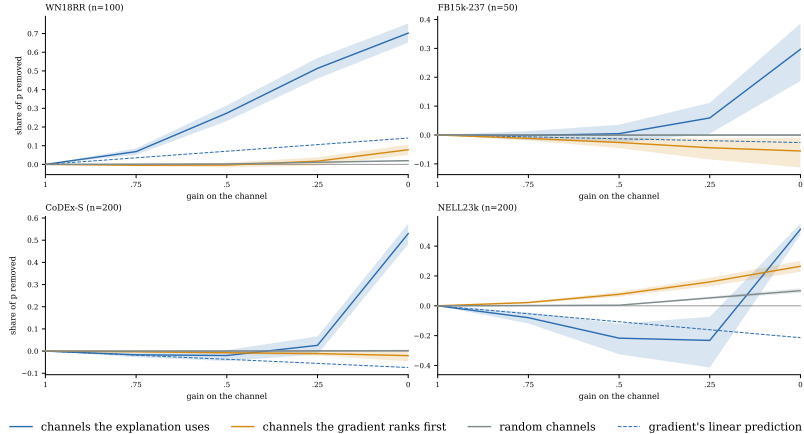
so a slow, low-dimensional **context** reshapes fast **content** without rewiring it.

ULTRA already works this way. Its relation pass runs once per query and gives one vector q_c per relation type. Every message along an edge of type c is $h \odot q_c$: one channel reaching all of its edges at once.

The pharmacology test

- Give each channel a gain g_c; $g = 1$ is the real model.
- Gradient $\partial p / \partial g_c$ at $g = 1$: the slope. It is NBFNet's own edge gradient, summed over the channel.
- Dose response: turn g_c down to .75, .5, .25, 0.
- Channels the explanation uses, channels the gradient ranks first, random channels as control.
- Check: gain 0 on the explanation's edges reproduces the deletion result exactly.

Result 4: the channels are switches, and the gradient cannot see it



Silencing the strongest channel the explanation uses removes 0.995, 0.770, 0.895, 0.895 of p (WN, FB, CX, NL); a random channel removes $\leq$0.101. At half gain only 0.29, 0.23, 0.19, 0.03 of that effect is there. The corrected gradient has the right sign 59.2, 64.9, 60.8, 73.9% of the time.

What it adds up to

The method **localises**. Localisation is not explanation.

- It finds a region of the graph the model depends on: true on all four graphs, against a random control that never moves.
- As released, it differentiates the wrong model. Fixed, it still does not rank within that region; integrated gradients rank better, still far from faithful.
- It does not decompose the region into separate evidence, and the list looks the same whether the model did multi-hop reasoning or one lookup.
- **Cheap fixes:** reverse the graph before `visualize()`; rank with integrated gradients; report the busiest edge and confirm it with one deletion; print the distinct-edge count.

Limits: one model family; correct predictions only; deleting a path disturbs paths that share its edges, which I measured and report as a finding rather than argue away.

Backup: the control I found by reading an example

Building a worked example, the top “path” came back as one hop: the stored reverse of a training fact. WN18RR keeps symmetric relations, so for 57.5% of its queries one lookup answers the question. The test edge itself is never present, so this is not leakage, but it changes what there is to explain.

WN18RR	reverse present	reverse absent
queries	115	85
paths sharing one edge	0.894	0.872
Kendall τ	-0.012	-0.003
one edge recovers	0.807	0.395

Redundancy and the dead ranking hold on both halves. How much one edge carries does not, which is why it is reported per graph and per half.